y.
Finding new customers willing to onboard on untested infrastructure turned out to be a challenge, especially with missing features.
Not having a customer in mind makes it impossible for us to define scope of which features to deliver.

Decision

Cells 1.0 will only be for internal customers of GitLab where a subset of team members will have their groups migrated to another Cell and work on that Cell.
This will allow us to dogfood important workflows before rolling them out to customers.
Groups and projects of the internal customer will have to be private, because the Organization will stay private.
Internal customers might not need things like the CI Catalog or Advanced Search.

The first internal customer to migrate is yet to be identified.
The migration will loosely follow this plan:

1. Create GitLab Inc. Organization on another Cell
1. Use Direct Transfer to move a group from the existing GitLab.com infrastructure to the other Cell.
1. Use Org Mover when it's ready to migrate the rest of the top-level groups and the feature set is enough for that top-level group. For example, gitlab-org will be moved in Cells 2.0.

Consequences

Users of Cells 1.0 can only be associated with one Organization, so certain GitLab team members will have to manage two accounts: one to access the default Organization on GitLab.com, and one to access GitLab Inc. on another Cell.
This might temporarily complicate their workflow, in case they want to collaborate with the rest of the GitLab team members.

Alternatives

The only alternative is finding new customers who are willing to partner with us.
This search is ongoing, and the above proposal will allow us to move forward regardless.

SECTION: engineering/architecture/design-documents/cells/decisions/008_database_sequences.md

Context

Having non-overlapping unique sequences across the cluster is necessary for moving organizations between cells,
this was highlighted in core-platform-section/-/epics/3
and different solutions were discussed in <https://gitlab.com/gitlab-org/core-platform-section/data-stores/-/issues/102>.

Decision

All cells will have bigint IDs on creation. While provisioning, each of them will get a range of sequences to use from the Topology Service.

Topology service uses the logic explained in here to compute the sequence range, which is used to set
minval, maxval for all existing and newly created sequence IDs.

- Once the database(s) are loaded, gitlab:db:altercellsequencesrange is called to alter sequences range.
- The above rake internally registers a EVENT TRIGGER alternewsequencesrange to set the correct sequence range for new IDs.

Alternatives

Below are the different solutions considered for this problem.

- Solution 1: Global Service to claim sequences
- Solution 2: Converting all int IDs to bigint to generate uniq IDs
- Solution 3: Using composite primary key [(existing PKs), original cell ID]
- Solution 4: Use bigint IDs only for Cell
- Solution 5: Using Logical replication

SECTION: engineering/architecture/design-documents/cells/decisions/009_cell_initial_sizing.md

Context

When we provision a Cell we have to choose a Reference Architectures to start with, then go on to scale accordingly to the workloads based upon flexible architecture to scale accordingly to the workloads.

In <https://gitlab.com/gitlab-com/gl-infra/scalability/-/issues/2838> we did some research on which reference architecture to choose initially.

Decision

Ring 0

For Ring 0 we will only run QA Jobs and it will not serve any customer traffic, for cost efficiency reasons we will go with a 3k reference architecture.

Ring 2 and above

The first Cell will be used for internal customers only/ GitLab Inc only, and it will be done gradually so that not all of GitLab Inc repositories will be moved at once.
The time between when we first provision this Cell vs when we on-board all of GitLab Inc is still unknown,
so we'll start with a medium sized Cell 25k reference architecture sized Cell, and then scale

it up to a 50k reference architecture.

The other Cells that we'll provision in this ring and other outer rings will start with a 50k reference architecture.

Consequences

For the Cell in Ring 0, we don't see any consequences yet.

For the Cell serving GitLab Inc we might need to be scale it from 25k to 50k which can result in downtime
because we need to resize stateful services like Gitaly, Database, and Redis.
The duration of this downtime is still unknown and untested,
but for Dedicated on AWS this is around 20-40 minutes.
If we have to take this downtime it will be only internally,
and can be done over the weekend for minimal impact.

Alternatives

For the first Cell serving GitLab Inc. we can go with a 50k reference architecture from the start but:

- We don't know if we'll need all these resources when we onboard everyone.
- We still don't know when we'll onboard everyone resulting in a lot of wasted compute.
- We can use flexible reference architectures to fix specific hot spots in the architecture first.

SECTION: engineering/architecture/design-documents/cells/decisions/010_http_router_rules_and_cache.md

Context

HTTP Router is integral part of the Cells 1.0 architecture. The definition of rules and usage of cache was a confusing part of the design. This change clarifies the behavior.

Decision

- HTTP Router does use only static rules that are part of project, or an router deployment.
- The rules are described in a JSON.
- The rules might define to be passthrough (use fixed address or from the config),
  or classify (use Topology Service).
- The classify requests do use HTTP headers to control cache behavior instead of json response.
- The usage of HTTP headers is compatible with caching behavior expected by the
  Cloudflare Workers - Cache.

Consequences

- The HTTP Router design is simplified as we don't need to resolve now how to merge routing

rules from many
  distinct Cells, possibly running different versions, and all complexities involved by this
note.
- The usage of HTTP cache control headers make it seamless to use with Cloudflare Workers.
- If the need for alternative methods for merging rules emerges or managing cache this
simplification
  is compatible with any future design improvement.

 Alternatives

- No alternatives were considered since it is a simplification of the original design.

SECTION: engineering/architecture/design-
documents/cells/decisions/011_cell_specific_configuration.md

Context

A Cell is provisioned using a Tenant Model that is consumed by Instrumentor.
Both Instrumentor and the Tenant Model are shared components between GitLab.com and GitLab
Dedicated,
and we need a way to split the code path for provisioning/configuring a Tenant for
configurations that will only ever be enabled for a Cell, for example:

- gitlabsession prefix
- Host configuration
- Disable GitLab Dedicated custom ApplicationSettings for Cells

 Decision

We will add a single field inside of the tenant model which will be cells.
cells will be an object, which will have keys specific to cell configuration,
for example:

json
```
{
  "tenantid" : "c01j2gdw0zfdafxr6"
  ...
  "cells": {
    "cellid": 1
  }
}
```

The cells.cellid will be a mandatory field, which will indicate that this tenant is a Cell.
By default cell is optional so existing tenant models will continue to work,
and new GitLab Dedicated tenants can omit it.

When cell.cellid is specified we'll follow convention over configuration,

and trigger all the extra configuration we need to do for a tenant to run as a cell, for example:

- gitlabsession prefix
- Host configuration
- Disable GitLab Dedicated custom ApplicationSettings for Cells
- Entitlement for PAM during onboarding,
  where we'll check if it's a Cell Tenant and add the correct groups.

The value of cells.cellid will come from the Topology Service and they need to match, since this will make sure the topology service is aware of that Cell so it can route traffic to it.

Consequences

- We follow convention over configuration where a single config field enables all of Cells configuration,
  without adding multiple configuration fields to the tenant model.
- We need to know the cells.cellid that will be used by the Topology Service
  before we can fully provision a Cell.
- We will only add new fields to cells object if that is our only option, and
  we can't create a convention with cellid.

Alternatives

- Create a new field cells: true that does the same thing, but we still need
  another configuration field for cellid, since we shouldn't couple
  cellid and tenantid. Coupling the two will have a few cons.
  - Using tenantid to session/PAT prefix will be too long.
  - Expose the tenantid to the user which is not ideal.
  - Couple application logic with tenantid which has different constrains and meanings.
- Create a field for each configuration field. This will end up exploding the
  tenant model configuration, and make it hard to know which fields are needed to
  make a tenant work for cells.
- Create new reference architecture for Cells. The reference architecture mainly
  specifies how big/small a GitLab Tenant is. It will overload the reference
  architecture meaning, and we still need specific values
  like cellid configurable somewhere.
- Using Reference Architecture Overlays
  has the same problem as the reference architecture point before this,
  it's about how big/small a GitLab a Tenant is going to be.

SECTION: engineering/architecture/design-documents/cells/decisions/012_cell_unique_identifier.md

Context

We will have multiple Cells, each of which has a subset of customer data in them.

This means we need a unique identifier for each Cell to Route, Claim, and create sequences.

Cell provisioning tooling already has Tenant ID
which is used as a unique identifier for all the infrastructure that is
provisioned.

Decision

Introduce the concept of Cell ID, where it will be an incrementing ID starting
from 1. This Cell ID will be used across multiple parts of the system:

- Topology Service: Used to classify resources, claim resources by a specific
  Cell ID, and create Database Sequence.
- Tenant Model: Used to configure the
  Rails Monolith with the correct Cell ID and enable Cell-specific configuration
  inside of Instrumentor.
- HTTP Router: Used as identifiable information to
  route the request to the correct Cell.
- Rails Monolith: When configured it will start claiming globally unique resources to Topology Service and create routable information like tokens

The Cell ID will have the following rules:

- Cell ID 1 will be reserved for the existing legacy Cell, and any new Cell provisioned will increment by 1, meaning the first we will provision will have a Cell ID of 2.
- Cell ID can only be used once, and new provisioned Cell should have a new ID.

Consequences

The Cell ID will need to be propagated through all the services that care about
Cells. For this to scale we need a single service that will be the single source
of truth for the Cell ID and where it points to.

Topology Service will be the owner and single source of truth of Cell ID, every
other system will use Topology Service to see how many Cells we have and the IDs
of those Cells.

Alternatives

1. Use Tenant ID as an identifier: This would couple the Rails Monolith,
   Topology Service, and HTTP Router with the provisioned infrastructure. Making
   it hard to move the immutable infrastructure to a new tenant model, in case we
   want to provision a brand new tenant with the same Cell ID.
1. Cell Name: Give a Cell a specific name, this name still needs to be unique
   and probably derived from an ID anyway. If we were to use a cell name, that
   would be purely to make it human-readable, but no system would use it as an
   identifier, since it would be hard to keep the ID and Name in sync.
   The name of the Cell should follow convention over configuration
   and follow the format of cell-{ID}.

SECTION: engineering/architecture/design-documents/cells/decisions/013_cell_restore_from_backup.md

Context

We discussed in this epic whether restoring a Cell from backup should be done in-place, or into a new Cell with the same or a different Cell ID.

Decision

It was decided that for Cells 1.0:

1. We will prioritize restoring into a new Cell rather than restoring in-place in an existing Cell.
1. When restoring a Cell, the recovered Cell will use the same Cell ID as the original Cell.
1. When restoring a Cell, the recovered Cell will have a different and unique Tenant ID.
1. If a recovered Cell will need to become permanent, an update will be made in the Topology service to update the address for the existing Cell ID, to point to the new Tenant.

There will be two modes of Cell restoration:

1. If the recovered cell will be permanent:
   1. Restrict writes or traffic to the Cell being recovered.
   1. Provision the recovered Cell with the same Cell ID, different Tenant ID.
   1. Register the recovered cell with the topology service with the new address.
1. If the recovered cell will be used for validating restore:
   1. Provision the recovered Cell with the same or unset Cell ID and a different Tenant ID.
   1. Don't add the recovered Cell to the topology service; it can't be assigned sequence ID or Claims.
   1. Validate the recovered Cell by connecting to it directly, bypassing the routing.
   1. Tear down the recovered Cell.

Note: During the recovery process we may need to utilize silent mode on the recovered Cell, or maintenance mode on the Cell being recovered.

Consequences

1. By prioritizing recovery into a new Cell, we can automate and test the restore process in Production.
1. Using the same or unset Cell ID on the restored Cell, external traffic will not be routed to it. The recovered Cell will not by default register itself with the Topology Service.
1. By assigning a new Tenant ID to the recovered Cell, it will allow us to validate the Cell by connecting directly to it, but not route traffic to it.

Alternatives

1. Restore affected components in place.
   We decided to de-prioritize this approach as it doesn't allow for easy validation in

Production of restore procedures.
2. Assign a new Cell ID to the recovered Cell.
   Because the recovered Cell may be the replacement for an existing Cell. For routing purposes, we want to keep the Cell ID the same.
   If Cell ID changes, we would need to re-write multiple claims in Topology Service making the recovery process slower and more compute intensive.

SECTION: engineering/architecture/design-documents/cells/decisions/014_clusterwide_syncing_in_cells_1_0.md

Context

In order for some features to work, the data for some
tables needs to be synchronized in some way to all cells.
For example, the plans, subscriptionaddons, and workitemtypes tables do need to be the same across all cells.

Decision

1. There will be no application layer clusterwide synchronization in Cells 1.0.
1. Static data tables, like plans do not need synchronization, but rather
   converted to be always consistent by being hard-coded in application code.
   A good example is
   VisibilityLevel.
1. Cell Setting tables, like applicationsettings can be set independently.
   An external source of truth like
   Terraform will
   propogate the desired values for each ring of cells.
   To support this, an internal API is required for each Cell Setting
   table.

Pros

1. We can avoid work to prevent writes on follower cells, and setup syncing service, and creating APIs to update each Static data type table.
1. We reduce consistency risks. If for any reason, the syncing fails, the application might
   start producing corrupt data. For example creating gitlabsubscriptions with bad
   planid values. If we replace planid column to use a globally unique
   reference instead, we will not have any consistency risk.
1. No need to create a leader/follower topology between cells/databases and no need to keep clusters in quorum.
1. Each cell is still fully isolated and doesn't depend on one another.

Cons

1. For Cell settings, we will need to tolerate a small amount of time where
   there may be configuration drift.
1. For Static data tables, there may be downstream services that depend on these

data. So we will need to wait for an application change to fully propogate through all rings first, before updating any downstream service to use the new data.

Analysis of clusterwide tables

An analysis of clusterwide tables was performed on 2025-01-13.
The result is that we can categorized into 4 different types:

1. Static data table. Tables which are constant / exactly the same for all cells.
1. Cell Setting table. Tables which host settings which needs to affect all cells.
1. Organization / Cell table. Tables which may be better categorized as gitlabmaincell.
1. User table. Tables related to users, and can be synchronized later in Cells 1.5+ (not Cells 1.0).

This section lists the full list of tables to the different types.

Cell Setting tables

applicationsettings

See related issue: issue 505685.

In short, we will use an external source of truth
to set each cell's Application Settings.

As a migration step, we will need to first obtain the current values from the Legacy Cell, to copy to our external source of truth.
The external source of truth will then propgate the value to each ring.

When creating a setting, developers need to ensure that the default for the setting will work correctly for any Cell.
This applies especially when the new setting has not had a chance to be set by the external source of truth.

broadcastmessages

Similar to applicationsettings, broadcast messages will be configured by an external source of truth.

The broadcastmessages.id column is referred to by the userbroadcastmessagedismissals table - it is used to record if a user has dismissed a broadcast message.
To improve consistency, the value of the broadcastmessages.id may be set directly via API.

Static data tables

Convert reference tables to be in application code instead.

plans

The plans table is a simple table with id, name, and title columns.
It also has a unique index on the name column.
There are two referencing tables, planlimits and gitlabsubscriptions.

The problem is that each Cell could create in-consistent data where
the name does not match id in all cells.

The solution is simple.
We need a globally unique reference for each plan.
We can have the following enum:

```ruby
  enum :nameuid,
    default: 1,
    free: 2,
    bronze: 3,
    silver: 4,
    premium: 5,
    gold: 6,
    ultimate: 7,
    ultimatetrial: 8,
    ultimatetrialpaidcustomer: 9,
    premiumtrial: 10,
    opensource: 11
```

And drop the id column.
We will then use the new nameuid column in all
referencing tables.

Another alternative is to drop the plans table entirely, and use a hard-coded
list of plans.

subscriptionaddons

The subscriptionaddons table is also a simple table with id, name, and
description columns.
Again, it has a unique index on the name column.

Similar to the plans, we can either use a nameuid column strategy, or drop
the table entirely.

workitemtypes

The workitemtypes table was re-organized to have a
constant set of IDs.

Unless there are reasons not to, we should convert further to use a hard-coded list of work item types.

abusereportlabels

This table abusereportlabels has several columns:

- id
- cachedmarkdownversion
- title
- color
- description
- descriptionhtml

There is a unique index for the title column.

There is no conceptual need to synchronize this table between each Cell.
Abuse reports are independent records.
abusereportlabels are labels which are attached to abuse reports.

The only problem arises when abusereportlabels are moved between Cells, leading to uniqueness violations for the title column.
The simplest measure is to drop the uniqueness constraint, and allow duplicates.

Alternatively, we can append (Cell 2) to the title to de-duplicate.

programminglanguages

The programminglanguages table is a table with id, name, and color columns.
The table has a unique index on the name column.

Similar to the plans, we can adopt the nameuid column strategy, and drop the id column.
As the data comes from Gitaly (linguist), we will need to map the name to an integer in a way that is stable.
This mapping can be stored on either the GitLab Ruby monolith, or in Gitaly.

<https://gitlab.com/gitlab-org/gitlab/-/blob/816e3ce6770ad96e3d5b0d7dae4925d63efa02fc/vendor/languages.yml> has the full list of languages.
We can possibly use the languageid field.

All referencing tables will be switched to refer to the nameuid column instead.

securitytrainingproviders

The securitytrainingproviders table has a few columns:

- id
- name
- description
- url
- logourl

There is a unique index on name.

Similar to the plans, we can adopt the nameuid column strategy, and drop the
id column.
All referencing tables will be switched to refer to the nameuid
column instead.

However, as there are only three rows, we can drop the table entirely instead,
and use in-application code instead.

ruby
SECUREFLAGDATA = {
  nameuid: 1,
  name: 'SecureFlag',
  description: "Get remediation advice with example code and recommended hands-on labs in a
fully
              interactive virtualised environment.",
  url: "https://knowledge-base-api.secureflag.com/gitlab"
}.freeze

 Organization / Cell tables

These tables are likely mis-categorized as gitlabmainclusterwide, and need
to be classificed as gitlabmaincell instead.

 User tables

These tables are related to the users table.

As Users can only exist on one cell in Cells 1.0, there is no need to
synchronize any user related data until Cells 1.5+

 Tables

This lists all clusterwide tables and its type.

| Table | Static data table | Cell Setting
table | Organization/cell table | User table | Rows Present in new GDK |
|--|-----------------|----------------
--------|-------------------------|------------|-------------------------|
| aifeaturesettings | | Y
| Maybe ? | | N |
| aisettings | | Y

		N	
appearances			Y
		N	
applicationsettings			Y
		Y	
cloudconnectoraccess			Y
		N	
planlimits			Y
		N	
serviceaccesstokens			Y
		N	
aiselfhostedmodels			Y
Maybe ?		N	
applicationsettingterms			Y
		N	
broadcastmessages			Y
Maybe ?		N	
licenses			Y
		N	
plans		Y	
		Y	
subscriptionaddons		Y	
		Y	
workitemhierarchyrestrictions		Y	
		Y	
workitemrelatedlinkrestrictions		Y	
		Y	
workitemtypes		Y	
		Y	
workitemwidgetdefinitions		Y	
		Y	
abuseevents			
	Y	N	
abusereportassignees			
	Y	N	
abusereportevents			
	Y	N	
abusereportlabellinks			
	Y	Y	
abusereportlabels			
	Y	Y	
abusereportnotes			
	Y	N	
abusereportusermentions			
	Y	N	
abusereports			
	Y	Y	
abusetrustscores			
	Y	N	
aitestingtermsacceptances			

Maybe		N	
atlassianidentities		
	Y	N	
auditeventsinstanceamazons3configurations		
Maybe		N	
auditeventsinstanceexternalauditeventdestinations		
Maybe		N	
auditeventsinstanceexternalstreamingdestinations		
Maybe		N	
auditeventsinstancegooglecloudloggingconfigurations		
Maybe		N	
auditeventsinstancestreamingeventtypefilters		
Maybe		N	
auditeventsstreaminginstanceeventtypefilters		
Maybe		N	
authenticationevents		
	Y	Y	
awsroles		
	Y	N	
bannedusers		
	Y	N	
deploytokens		
	Y	N	
earlyaccessprogramtrackingevents		
	Y	N	
emails		
	Y	Y	
ghostusermigrations		
	Y	N	
gpgkeysubkeys		
	Y	N	
gpgkeys		
	Y	N	
identities		
	Y	N	
instanceauditevents		
Maybe		N	
instanceauditeventsstreamingheaders		
Maybe		N	
instanceintegrations		
Maybe		N	
keys		
	Y	Y	
oauthapplications		
Maybe		N	
programminglanguages	Y	
Maybe		Y	
redirectroutes		
Y		N	
routes		

Y		Y	
savedreplies		
	Y	N	
securitytrainingproviders		Y	
Maybe		Y	
smartcardidentities		
	Y	N	
spamlogs		
	Y	Y	
termagreements		
	Y	N	
useragentdetails		
	Y	N	
userauditevents		
	Y	Y	
userbroadcastmessagedismissals		
	Y	N	
usercallouts		
	Y	N	
usercreditcardvalidations		
	Y	N	
usercustomattributes		
	Y	N	
userdetails		
	Y	Y	
userfollowusers		
	Y	N	
userhighestroles		
	Y	N	
usermemberroles		
	Y	N	
userpermissionexportuploads		
	Y	N	
userphonenumbervalidations		
	Y	N	
userpreferences		
	Y	Y	
userstatuses		
	Y	N	
usersyncedattributesmetadata		
	Y	N	
users		
	Y	Y	
usersstatistics		
	Y	N	
vscodesettings		
	Y	N	
webauthnregistrations		
	Y	N	

SECTION: engineering/architecture/design-documents/cells/diagrams/_index.md

Diagrams used in Cells are created with draw.io.

Edit existing diagrams

Load the .drawio.png or .drawio.svg file directly into draw.io, which you can use in several ways:

- Best: Use the draw.io integration in VS Code.
- Good: Install on MacOS with brew install drawio or download the draw.io desktop.
- Good: Install on Linux by downloading the draw.io desktop.
- Discouraged: Use the draw.io website to load and save files.

Create a diagram

To create a diagram from a file:

1. Copy existing file and rename it. Ensure that the extension is .drawio.png or .drawio.svg.
1. Edit the diagram.
1. Save the file.
1. Optimize images with pngquant -f --ext .png .drawio.png to reduce their size by 2-3x.

To create a diagram from scratch using draw.io desktop:

1. In File > New > Create new diagram, select Blank diagram.
1. In File > Save As, select Editable Bitmap .png, and save with .drawio.png extension.
1. To improve image quality, in File > Properties, set Zoom to 200%.
1. To save the file with the new zoom setting, select File > Save.
1. Optimize images with pngquant -f --ext .png .drawio.png to reduce their size by 2-3x.

DO NOT use the File > Export function. The diagram should be embedded into .png for easy editing.

SECTION: engineering/architecture/design-documents/cells/impacted_features/admin-area.md

{{% alert %}}
This document is a work-in-progress and represents a very early state of the
Cells design. Significant aspects are not documented, though we expect to add
them in the future. This is one possible architecture for Cells, and we intend to
contrast this with alternatives before deciding which approach to implement.
This documentation will be kept even if we decide not to implement this so that
we can document the reasons for not choosing this approach.
{{% /alert %}}

In our Cells architecture proposal we plan to share all admin related tables in GitLab.

This allows for simpler management of all Cells in one interface and reduces the risk of settings diverging in different Cells.
This introduces challenges with Admin Area pages that allow you to manage data that will be spread across all Cells.

1. Definition

There are consequences for Admin Area pages that contain data that span "the whole instance" as the Admin Area pages may be served by any Cell or possibly just one Cell.
There are already many parts of the Admin Area that will have data that span many Cells.
For example lists of all Groups, Projects, Topics, Jobs, Analytics, Applications and more.
There are also administrative monitoring capabilities in the Admin Area that will span many Cells such as the "Background Jobs" and "Background migrations" pages.

2. Data flow

3. Proposal

We will need to decide how to handle these exceptions with a few possible options:

1. Move all these pages out into a dedicated per-Cell admin section. Probably the URL will need to be routable to a single Cell like /cells/<cellid>/admin, then we can display these data per Cell. These pages will be distinct from other Admin Area pages which control settings that are shared across all Cells. We will also need to consider how this impacts self-managed customers and whether, or not, this should be visible for single-Cell instances of GitLab.
1. Build some aggregation interfaces for this data so that it can be fetched from all Cells and presented in a single UI. This may be beneficial to an administrator that needs to see and filter all data at a glance, especially when they don't know which Cell the data is on. The downside, however, is that building this kind of aggregation is very tricky when all Cells are designed to be totally independent, and it does also enforce stricter requirements on compatibility between Cells.

The following overview describes at what level each feature contained in the current Admin Area will be managed:

Feature	Cluster	Cell	Organization
Abuse reports			
Analytics			
Applications			
Deploy keys			
Labels			
Messages	·		
Monitoring		·	
Subscription			
System hooks			
Overview			

Settings - General	·		
Settings - Integrations	·		
Settings - Repository	·		
Settings - CI/CD (1)	·	·	
Settings - Reporting	·		
Settings - Metrics	·		
Settings - Service usage data		·	
Settings - Network	·		
Settings - Appearance	·		
Settings - Preferences	·		

(1) Depending on the specific setting, some will be managed at the cluster-level, and some at the Cell-level.

4. Evaluation

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/advanced-search.md

{{% alert %}}
This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.
{{% /alert %}}

Summary

The Advanced search functionality
allows users to search across the entire GitLab instance. Advanced search supports Elasticsearch and OpenSearch as the search backend.

NOTE: Global search is also an impacted feature that will be covered in a separate design document.

GitLab.com has one Elasticsearch cluster that houses all indexed data to support Advanced search. Advanced search includes an automated indexing pipeline and migration framework
for data migrations. Most infrastructure and index maintenance tasks are performed manually by the Global Search team through the change request workflow.

When we introduce multiple Cells, Advanced Search will need to support:

1. Infrastructure maintenance
   - Elasticsearch cluster version upgrades
   - Scaling the Elasticsearch cluster
   - Support for incident root cause analysis and resolution
2. Index maintenance
   - Index shard resizing using Zero-downtime reindexing feature (example issue)
   - Index shard resizing using Split shards Elasticsearch API (example issue)
   - Enabling the Elasticsearch slow log (example issue)
   - Cleaning up reverted migrations from the Advanced search migrations index (example issue)
3. Organization migration
   - Advanced search will need to support organizations being moved from one cell to another.
   - When an organization is moved, we need to re-index database records, embeddings, and git data.

This design document explains the high-level plan for how we will support these areas.

Proposal

Each cell will have its own Elasticsearch cluster.

Infrastructure maintenance

All infrastructure maintenance must be automated. We may consider using the migration framework, but other methods should be evaluated.

To support this, the following is required:

- The indexing paused framework must be modified to allow pausing indexing for a specific index.
  Pausing for a specific index allows maintenance tasks to be performed on an index without impacting other data types. This is important to reduce impact of search results being out of date because indexing is behind.
- Search and indexing validations must be automated. The Search::ClusterHealthCheck class can be expanded to include search and indexing validation. gitlab499586

Index maintenance

All index maintenance tasks will be initially be completed using the Advanced search migration framework.
Once the migration process is verified, all index maintenance migrations will be migrated by Cron workers. There should be a Cron worker per maintenance task:

1. Shard resize Cron worker will review each index and adjust shard size if over the threshold
1. Migration cleanup Cron worker will remove migrations from the migrations index that do not exist in codebase

1. Slow log Cron worker will enable the slow log if an ops feature flag (or application setting) is enabled

Organization migration

There are multiple ways to approach organization migration and each comes with a set of trade-offs. Since Cells is being rolled out in phases, this proposal contains recommendations by phase.

[Cells 1.0] Use existing indexing framework

Organization migration will be automated by using the existing indexing framework. Once an organization is put into maintenance mode and all PostgreSQL and Gitaly data has been moved, the organization can be queued for indexing in the destination cell.

Conduct benchmarks to determine how long organizations at pre-determined sizes take to migrate. This data will be used to determine if the indexing pipeline will continue to be used for organization migration in later cells phases.

To support this, the following is required:

- The framework needs to be adjusted to handle indexing an organization. This may be as simple as iterating through all namespaces for an organization and kicking off the appropriate workers.

Benefits of this approach:

1. Cells can use any search cluster type and version.
1. Existing framework is well tested and understood by team members.

Downsides of this approach:

1. Indexing may take too long for organizations. We need to benchmark different organization sizes. Related issues:
   1. gitlab480372
   1. gitlab391489
1. There are no indicators when indexing is complete for database records. Organizations will have missing search results until indexing is complete.
1. Additional load on GitLab instance including Sidekiq, Gitaly, and PostgreSQL. Other Sidekiq jobs in the cell will impact indexing completion.

[Cells 2.0] Use Elasticsearch APIs to move data

The plan below is a proposal only. The search platform for Cells has not been decided at this time. If the search platform will be different between Cells, the proposal will need to be updated.

If benchmarking results show the existing indexing framework is not fast enough, organization migration could be automated with the Elasticsearch Reindex API

to move all indexed data for an organization to the new cell's Elasticsearch cluster. The API supports moving data between two different clusters (or hosts).

To support this, the following is required:

- The indexing paused framework must be modified to ensure that all queued indexing requests are completed once an organization is put into "maintenance mode". This is needed to ensure all indexed data is up to date before the migration.
- New queues should be setup to work on migrated data only. This will prevent other active indexing jobs from taking priority in the existing queues.
- Database ID values must remain consistent between cells. Cluster wide unique database sequences are planned for Cells.

Proposed workflow:

mermaid
flowchart TD
    A[Organization scheduled for migration] -->B
    B(Organization put into maintenance mode) -->C
    C(Complete all queued indexing requests for the organization) -->D
    D(Enqueue a worker to reindex all organization data from source cell Elasticsearch cluster to destination cell Elasticsearch cluster) -->|1-3 days in future|E
    E(Cleanup organization data from source cell)

Benefits of this approach:

1. No additional load on GitLab instance including Sidekiq, Gitaly, and PostgreSQL.
1. Know exactly when indexing is complete.

Downsides of this approach:

1. Requires all search clusters to be on the same platform.
1. Requires allowed communication between cell search clusters.

SECTION: engineering/architecture/design-documents/cells/impacted_features/agent-for-kubernetes.md

{{% alert %}}
This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.
{{% /alert %}}

TL;DR

SECTION: engineering/architecture/design-documents/cells/impacted_features/backups.md

{{% alert %}}
This document is a work-in-progress and represents a very early state of the
Cells design. Significant aspects are not documented, though we expect to add
them in the future. This is one possible architecture for Cells, and we intend to
contrast this with alternatives before deciding which approach to implement.
This documentation will be kept even if we decide not to implement this so that
we can document the reasons for not choosing this approach.
{{% /alert %}}

Each Cell will take its own backups, and consequently have its own isolated backup/restore procedure.

1. Definition

GitLab backup takes a backup of the PostgreSQL database used by the application, and also Git repository data.

2. Data flow

Each Cell has a number of application databases to back up (for example, main, and ci).
Additionally, there may be cluster-wide metadata tables (for example, users table) which is directly accessible via PostgreSQL.

3. Proposal

3.1. Cluster-wide metadata

It is currently unknown how cluster-wide metadata tables will be accessible.
We may choose to have cluster-wide metadata tables backed up separately, or have each Cell back up its copy of cluster-wide metadata tables.

3.2 Consistency

3.2.1 Take backups independently

As each Cell will communicate with each other via API, and there will be no joins to the users table, it should be acceptable for each Cell to take a backup independently of each other.

3.2.2 Enforce snapshots

We can require that each Cell take a snapshot for the PostgreSQL databases at around the same time to allow for a consistent enough backup.

4. Evaluation

As the number of Cells increases, it will likely not be feasible to take a snapshot at the same time for all Cells.
Hence taking backups independently is the better option.

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/ci-cd-catalog.md

{{% alert %}}
This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.
{{% /alert %}}

The CI/CD pipeline components catalog is a currently experimental feature that aims at helping users reuse pipeline configurations.
Potentially, there are several aspects of the CI/CD catalog that might be affected by Cells:

1. Namespace catalog, exists today as an experimental feature. With the introduction of Cells we would likely remove the namespace catalog and create a single Organization catalog, where all users would see all available published components in a single place (based on their permissions). This would replace today's offering, where users can have multiple catalogs which are bound to a namespace and completely isolated from each other.
1. The community catalog is supposed to allow users to search across different Organizations.

1. Definition

The CI/CD pipeline components catalog makes reusing pipeline configurations easier and more efficient.
It provides a way to discover and reuse pipeline constructs, allowing for a more streamlined experience.

There are several flavors of the CI/CD catalog:

1. Namespace catalog (experimental): Bound to the top-level namespace (group or personal namespace). The namespace catalog aggregates all published components from Projects it contains. The number of top-level namespaces available in an Organization could potentially be the number of available catalogs.
1. Instance-wide component catalog (planned): Surfacing all the components that are scattered across an instance. All published components in a public or internal Project will be available in the instance-wide catalog. Only a single instance-wide catalog is planned per instance.
1. Community catalog (planned): Allow users to search all published components in different repositories across multiple namespaces. The original plan was to introduce a community catalog within self-managed customer that would act as an aggregator of all published components hosted in that instance.

2. Data flow

3. Proposal

Moving to Organizations is a great opportunity to improve the user experience and to reach parity for both self-managed and GitLab.com users.

- We introduce an Organization catalog which aggregates and surfaces all the published components that are hosted in a single Organization. The Organization catalog would make the namespace catalog obsolete.
- Once Organizations exist, GitLab.com users would need a community catalog to surface components across multiple Organizations. We need additional research to understand if such a solution is needed for self-managed customers as well.

4. Evaluation

Moving to a single Organization will improve the experience for users of the CI/CD component catalog.
Today we can have multiple catalogs based on the number of namespaces, making it difficult for users to surface information across an Organization.

4.1. Pros

- An Organization catalog will be one unified catalog serving as the single source of truth for an Organization.
- An Organization catalog would serve both self-managed and GitLab.com users, whereas the current plan was to introduce two types of catalogs: an instance-wide component catalog for self-managed and a community catalog for GitLab.com.

4.2. Cons

- A separate catalog that surfaces components across Organizations would need to be implemented to serve the wider community (community catalog). This catalog will be required for GitLab.com only, later on we can evaluate if a similar catalog is needed for self-managed customers.

SECTION: engineering/architecture/design-documents/cells/impacted_features/ci-runners.md

{{% alert %}}
This document is a work-in-progress and represents a very early state of the
Cells design. Significant aspects are not documented, though we expect to add
them in the future. This is one possible architecture for Cells, and we intend to
contrast this with alternatives before deciding which approach to implement.
This documentation will be kept even if we decide not to implement this so that
we can document the reasons for not choosing this approach.
{{% /alert %}}

GitLab executes CI jobs via GitLab Runner, very often managed by customers in their infrastructure.
All CI jobs created as part of the CI pipeline are run in the context of a Project.
This poses a challenge how to manage GitLab Runners.

1. Definition

There are 3 different types of runners:

- Instance-wide: Runners that are registered globally with specific tags (selection criteria)
- Group runners: Runners that execute jobs from a given top-level Group or Projects in that Group
- Project runners: Runners that execute jobs from one Projects or many Projects: some runners might
  have Projects assigned from Projects in different top-level Groups.

This, alongside with the existing data structure where cirunners is a table describing all types of runners, poses a challenge as to how the cirunners should be managed in a Cells environment.

2. Data flow

GitLab runners use a set of globally scoped endpoints to:

- Register a new runner via registration token https://gitlab.com/api/v4/runners
  (subject for removal) (registration token)
- Create a new runner in the context of a user https://gitlab.com/api/v4/user/runners (runner token)
- Request jobs via an authenticated https://gitlab.com/api/v4/jobs/request endpoint (runner token)
- Upload job status via https://gitlab.com/api/v4/jobs/:jobid (build token)
- Upload trace via https://gitlab.com/api/v4/jobs/:jobid/trace (build token)
- Download and upload artifacts via https://gitlab.com/api/v4/jobs/:jobid/artifacts (build token)

Currently three types of authentication tokens are used:

- Runner registration token (subject for removal)

- Runner token representing a registered runner in a system with specific configuration (tags, locked, etc.)
- Build token representing an ephemeral token giving limited access to updating a specific job, uploading artifacts, downloading dependent artifacts, downloading and uploading container registry images

Each of those endpoints receive an authentication token via header (JOB-TOKEN for /trace, token all other endpoints).

Since the CI pipeline would be created in the context of a specific Cell, it would be required that pick of a build would have to be processed by that particular Cell.
This requires that build picking depending on a solution would have to be either:

- Routed to the correct Cell for the first time
- Be two-phased: Request build from global pool, claim build on a specific Cell using a Cell specific URL

3. Proposal

3.1. Authentication tokens

Even though the paths for CI runners are not routable, they can be made routable with these two possible solutions:

- The https://gitlab.com/api/v4/jobs/request uses a long polling mechanism with a ticketing mechanism (based on X-GitLab-Last-Update header). When the runner first starts, it sends a request to GitLab to which GitLab responds with either a build to pick by runner, or a 204 No job with a lastupdate token. This value is completely controlled by GitLab. This allows GitLab to use JWT or any other means to encode a cell identifier that could be easily decodable by Router.
- The majority of communication (in terms of volume) is using build token, making it the easiest target to change since GitLab is the sole owner of the token that the runner later uses for a specific job. There were prior discussions about not storing the build token but rather using a JWT token with defined scopes. Such a token could encode the cell to which the Router could route all requests.

3.2. Request body

This statement is no longer true:

> The most used endpoints pass the authentication token in the request body.
> It might be desired to use HTTP headers as an easier way to access this information
> by Router without a need to proxy requests.

The runner now sends the token in the HTTP headers in all requests with the
Add runner token to header MR merged.

3.3. Instance-wide are Cell-local

We can pick a design where all runners are always registered and local to a given Cell:

- Each Cell has its own set of instance-wide runners that are updated at its own pace
- The Project runners can only be linked to Projects from the same Organization, creating strong isolation.
- In this model the cirunners table is local to the Cell.
- In this model we would require the above endpoints to be scoped to a Cell in some way, or be made routable. It might be via prefixing them, adding additional Cell parameters, or providing much more robust ways to decode runner tokens and match it to a Cell.
- If a routable token is used, we could move away from cryptographic random stored in database to rather prefer to use JWT tokens.
- The Admin Area showing registered runners would have to be scoped to a Cell.

This model might be desired because it provides strong isolation guarantees.
This model does significantly increase maintenance overhead because each Cell is managed separately.
This model may require adjustments to the runner tags feature so that Projects have a consistent runner experience across Cells.

3.4. Instance-wide are cluster-wide

Contrary to the proposal where all runners are Cell-local, we can consider that runners are global, or just instance-wide runners are global.

However, this requires significant overhaul of the system and we would have to change the following aspects:

- The cirunners table would likely have to be decomposed into ciinstancerunners, ...
- All interfaces would have to be adopted to use the correct table.
- Build queuing would have to be reworked to be two-phased where each Cell would know of all pending and running builds, but the actual claim of a build would happen against a Cell containing data.
- It is likely that cipendingbuilds and cirunningbuilds would have to be made cluster-wide tables, increasing the likelihood of creating hotspots in a system related to CI queueing.

This model is complex to implement from an engineering perspective.
Some data are shared between Cells.
It creates hotspots/scalability issues in a system that might impact the experience of Organizations on other Cells, for instance during abuse.

3.5. GitLab CI Daemon

Another potential solution to explore is to have a dedicated service responsible for builds queueing, owning its database and working in a model of either sharded or Cell-ed service.
There were prior discussions about CI/CD Daemon.

If the service is sharded:

- Depending on the model, if runners are cluster-wide or Cell-local, this service would have to fetch data from all Cells.

- We could adapt a model of sharing a database containing cipendingbuilds/cirunningbuilds with the service.
- We could consider a push model where each Cell pushes to CI/CD Daemon builds that should be picked by runner.
- The sharded service would be aware which Cell is responsible for processing the given build and could route processing requests to the designated Cell.

If the service is Cell-ed:

- All expectations of routable endpoints are still valid.

In general usage of CI Daemon does not help significantly with the stated problem.
However, this offers a few upsides related to more efficient processing and decoupling model: push model and it opens a way to offer stateful communication with GitLab runners (ex. gRPC or Websockets).

4. Evaluation

Considering all options it appears that the most promising solution is to:

- Use Instance-wide are Cell-local
- Refine endpoints to have routable identities (either via specific paths, or better tokens)

Another potential upside is to get rid of cibuilds.token and rather use a JWT token that can much better and easier encode a wider set of scopes allowed by CI runner.

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/container-registry.md

{{% alert %}}
This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.
{{% /alert %}}

GitLab Container Registry is a feature allowing to store Docker container images in GitLab.

1. Definition

GitLab container registry is a complex service requiring usage of PostgreSQL, Redis and Object Storage dependencies.

Right now there's undergoing work to introduce Container Registry Metadata to optimize data storage and image retention policies of container registry.

GitLab container registry is serving as a container for stored data, but on its own does not authenticate docker login.
The docker login is executed with user credentials (can be personal access token) or CI build credentials (ephemeral cibuilds.token).

Container Registry uses data deduplication.
It means that the same blob (image layer) that is shared between many Projects is stored only once.
Each layer is hashed by sha256.

The docker login does request a JWT time-limited authentication token that is signed by GitLab, but validated by container registry service.
The JWT token does store all authorized scopes (container repository images) and operation types (push or pull).
A single JWT authentication token can have many authorized scopes.
This allows container registry and client to mount existing blobs from other scopes.
GitLab responds only with authorized scopes.
Then it is up to GitLab container registry to validate if the given operation can be performed.

The GitLab.com pages are always scoped to a Project.
Each Project can have many container registry images attached.

Currently, on GitLab.com the actual registry service is served via https://registry.gitlab.com.

The main identifiable problems are:

- The authentication request (https://gitlab.com/jwt/auth) that is processed by GitLab.com.
- The https://registry.gitlab.com that is run by an external service and uses its own data store.
- Data deduplication. The Cells architecture with registry run in a Cell would reduce efficiency of data storage.

2. Data flow

The data flow for docker login is documented in the container registry project. The equivalent requests using curl are documented below.

2.1. Authorization request that is send by docker login

shell
```
curl \
  --user "username:password" \
  "https://gitlab/jwt/auth?clientid=docker&offlinetoken=true&service=containerregistry&scope=repository:gitlab-org/gitlab-build-images:push,pull"
```

Result is encoded and signed JWT token. Second base64 encoded string (split by .) contains JSON

with authorized scopes.

json
{"authtype":"none","access":[{"type":"repository","name":"gitlab-org/gitlab-build-images","actions":["pull"]}],"jti":"61ca2459-091c-4496-a3cf-01bac51d4dc8","aud":"containerregistry","iss":"omnibus-gitlab-issuer","iat":1669309469,"nbf":166}

2.2. Docker client fetching tags

shell
```
curl \
  -H "Accept: application/vnd.docker.distribution.manifest.v2+json" \
  -H "Authorization: Bearer token" \
  https://registry.gitlab.com/v2/gitlab-org/gitlab-build-images/tags/list

curl \
  -H "Accept: application/vnd.docker.distribution.manifest.v2+json" \
  -H "Authorization: Bearer token" \
  https://registry.gitlab.com/v2/gitlab-org/gitlab-build-images/manifests/danger-ruby-2.6.6
```

2.3. Docker client fetching blobs and manifests

shell
```
curl \
  -H "Accept: application/vnd.docker.distribution.manifest.v2+json" \
  -H "Authorization: Bearer token" \
  https://registry.gitlab.com/v2/gitlab-org/gitlab-build-images/blobs/sha256:a3f2e1afa377d20897e08a85cae089393d
```